**OBJECT ORIENTED PROGRAMMING**
**Winter semester of acad. Year 2019-2020**
**Course Assignment**

The present assignment has as its subject the representation of a card exchange game (collectible/trading card game) between players[1], specifically a very simplified version of the Legend of the Five Rings (L5R)[2]. In games of this kind, the players collect cards (with information, statistics, images, decoration etc on the front side and a background design on the back) which they organize in piles and areas of the table (decks & areas). These cards are utilized strategically, following the rules and the phases of the respective game, with the aim of achieving one or more goals (as a rule, the reduction of some "health/dominance" points of the opponents).

# Game description

The game is played on a table (**GameBoard**) in which players (**Player**) participate. Each player represents an honourable (or... dishonourable) ruler who manages his (Japanese-style) dynasty. Each player keeps/holds his cards in predetermined areas of the table depending on their function or their state, for example he uses two decks of cards to draw new cards, his "hand" to activate cards in a specific phase of the game, four (4) cards that represent provinces of his dynasty etc. The goal of the player is the destruction of the provinces of the other players and the safekeeping of at least one of his own.

The one deck, called fate deck, contains green cards that represent actions available to the ruler (use of items and followers) and which the ruler keeps in his hand and chooses in the relevant phase of the game. The other deck, called dynasty deck, contains black cards which represent great personalities or holdings that are part of the dynasty and that are placed in an area in front of the player. Each deck has a size of at least forty (40) cards as well as a predetermined limit (usually 60 cards). In his hand he can hold up to a (predetermined, usually 6) maximum number of cards. Each ruler is characterized by his family honour (*honour*) which is initially given by a special card that represents his fortress (*Stronghold*).

The game is executed in rounds of multiple phases, while the players play in descending order based on their honour.

You are called upon to model the game and to implement its basic functions. The implementation of the playing process is NOT required, but the implementation of the relevant functions/operations (as they will be defined in the section "***Phases description***") is necessary so that the modelling is directly usable and makes possible the implementation of playing strategies and phase alternation.

You are, of course, welcome to define simple strategies of playing and of controlling the flow of a game, either automated or manual.

1 You can watch or play for free online at the address: https://jigoku.online/

2 Detailed, official rule guides: https://www.fantasyflightgames.com/en/products/legend-of-the-five-rings-the-card-game/.
**How to play**: https://www.youtube.com/watch?v=i9Mpjaobv4o

## *1.1 Table description (GameBoard)*

The first operation at the start of the program is the creation and initialization of the gameboard (*initializeGameBoard*), where the players who participate in the game are defined as well as the number of the various categories of cards that the players are allowed to have in their possession and that will be produced by the corresponding auxiliary functionality (deckbuilder). After the initialization of these, the initial state and the rest of the statistics of the game are printed (*printGameStatistics*) and the playing is executed (*gameplay*).

## *1.2 Cards description*

The game is played with cards of different types which define all the elements of the game. Each player "brings" to the game a deck (**DeckBuilder**) which contains all the cards that he can potentially use in the game. The number of each kind of card is agreed upon before the start of the game.

Each card (**Card**) has a specific name (*name*), cost (*cost*) and state (*isTapped*).
There are two groups of cards, the green ones (**GreenCard**) and the black ones (**BlackCard**).

The green ones can be of the follower category (**Follower)** or of the item category (**Item)**. These two categories of cards represent the personal guard and the equipment of a personality of the game. Each green card additionally has bonus attack points (*attackBonus*) and defence points (*defenceBonus*) that can be given to the corresponding character, a minimum value of required honour of the character (*minimumHonour*) and card text (*cardText*). Additionally, the player can activate an upgrade of the above bonus (*effectBonus*) by paying the relevant activation bonus (*effectCost*) during its purchase. The items, in addition to the above, also have durability (*durability*).

The black cards are either personalities (**Personality)** or holdings (**Holding)**. Each black card is revealed or hidden (*isReleaved*). The personalities additionally have attack (*attack*), defence (*defence*), honour (*honour*), can be alive or dead (*isDead*), while they can have followers and/or items can belong to them. A holding additionally has an amount of money that it yields per round (*harvestValue*), while it can be part of some other holding (*upperHolding*, *subHolding*).

The fortress (**Stronghold**) is a black card which does not participate in a deck but gives the player his initial honour (*honour*), the initial amount of money with which he starts the game (*money*) as well as the defence that each of his provinces has (*initialDefense*).

## 2.1 Kinds per card category

**Green cards :**

There are 5 kinds of items:
Katana, Spear, Bow, Ninjato, Wakizashi.
There are 6 kinds of followers:
Footsoldier, Archer, Cavalry, Bushido, Sieger, Atakebune/Naval.

**Black cards :**

There are 5 kinds of personalities:
Attacker, Defender, Champion, Chancellor, Shogun.
There are 7 kinds of holding:
Plain, Mine, Gold mine, Crystal Mine, Farmland, Gift&Favour, Stronghold.

## 2.2 Holdings chain description.

### *2.2.1 Chain Creation*

The holdings that belong to the categories Mine, Gold Mine, Crystal Mine can form a chain among them (be connected) and yield more money to their owners. Each holding can be connected only with one holding of lower value (*subHolding*) and only with one holding of higher value (*upperHolding*).

A connection is made during the purchase of a holding (see Purchase Phase). When a player buys a holding then all the holdings he has in his possession are searched, in order to determine whether there is an available holding with which a connection is possible. If there is an available one (*upperHolding* or *subHolding*) then the holding is connected with it and binds it. If there are more than one available, one is chosen at random. If there are two holdings which can be connected and complete the chain (i.e. the one being *subHolding* and the other *upperHolding*) then both are connected with the purchased one and are bound.

A holding that lies in the chain can be connected only with a compatible holding (*upperHolding*, *subHolding* or both). In the next section we analyze the structure and the rules by which a chain is created as well as the bonuses that it yields.

### *2.2.2 Chain creation rules and provided bonuses*

When a holding of the Mine category is connected with a Gold mine (*upperHolding*) then it yields an additional 2 money points to the total amount that it provides to the user.

If a holding of the Gold mine category is connected with a Mine (*subHolding*) then it yields an additional 4 money points to the total amount of money that it provides to the user. If the Gold mine has been connected only with a Crystal Mine (*upperHolding*) then it yields an additional 5 money points, while if it has been connected with a Mine and a Crystal Mine then it provides the player with additional money points equal to double the money that it initially gave to the player.

If the Crystal Mine has been connected with a Gold mine then it yields additional money points equal to the money that it initially provides to the user. If the chain Mine <-> Gold Mine <-> Crystal Mine has been completed, the Crystal mine provides additional money points equal to triple the money that it initially gave to the player.

Holdings of type Mine do not have a *subHolding* while holdings of type Crystal Mine do not have an *upperHolding*. All the remaining categories of holdings (that is except for the three types of mines) do not enter a chain.

### 2.2.3 Followers and Items Bonus

Each personality can have up to an upper limit of followers and items in its possession. Their attachment is done during the game, in the equipment phase. Whenever a battle is executed, the attack and defence points of the personality are increased by the bonus attack/defence points of the followers/items.

When the attachment of a follower or item is executed, one and only chance is given to the player to pay the upgrade price of the card so that an additional bonus of points is given.

## *2.3 Players description (Player)*

Each player has two main collections of cards: the green cards (fateDeck), from which he draws from the top of the deck whenever needed, as well as the black cards (dynastyDeck), which correspond to the provinces that he controls (provinces, of number numberOfProvinces). Each player also holds some green cards in his hand (hand), can have holdings under his possession (holdings) and also can control active personalities that are used for war (army). These collections are produced by random creation of cards (**DeckBuilder**) that follow initial limits, which are defined during the creation of the game.

The provinces (*provinces*) that the player controls are initially 4 and correspond to his “life points”. If the provinces of a player are all destroyed then he loses the game (see Winner of the game). Also the player can buy personalities and holdings from the black cards that are visible in his provinces.

## *2.4 Game phases description*

In each round we have 5 phases that are executed successively. Each phase is played by all the players in descending order of honour. Before the rounds of the game begin, each player starts with an initial number of green cards in his hand, without army, without holdings (except for his fortress) and with 4 provinces in a face-down position (not visible).

At the beginning of each round, all the cards are returned to the upright position (become untapped) and are available for use. Once a card goes to the sideways position (becomes tapped), it is used and the player cannot use it again for the rest of the round. For the execution of a purchase tapping is done to as many holdings as needed to cover the cost (any extra difference constitutes “lost” money).

### 2.4.1 Starting phase

In the starting phase (*startingPhase*), the player:

- returns all his cards to the upright position (*untapEverything*),
- draws a green card (*drawFateCard*),
- reveals the available provinces (*revealProvinces*)
- prints his current hand (*printHand*) and the provinces (*printProvinces*)

### 2.4.2 Equipment phase

Provided the player has active personalities (i.e. army), he can equip them with green cards from his hand (*equipPhase*). For this reason, the available cards of the player as well as his army are printed so that he can choose his moves. Provided the player has a sufficient amount of money, he can buy cards (reducing his money for the round) and equip them to the personality of his choice (always according to the maximum limit of each category of green cards per personality but also the minimum honour requirement that the followers and the items have). During the purchase of a card, he can choose to upgrade it by paying the relevant cost. If, in the end, the money is not enough or the personality is not sufficiently honourable, the relevant purchase is not carried out.

### 2.4.3 Battle phase

The player can activate personalities to attack either some province of an opponent, or let them be utilized for defence in a possible attack of the opponent (*battlePhase*). Only the upright personalities (*untapped*) can attack (*attack*) or defend (*defend*). In case of battle the sum of attack points of the attacking personalities is calculated and is compared with the sum of defence points of the defending cards and of the initial defence that the (under attack) province has. In case the total attack points of the attacking personalities minus the defence points of the defending cards (personalities and province) is greater than the defence of the defending province, then the attacker wins the battle, the province is destroyed (the total provinces of the defender are reduced by one) and the defender loses all his defending personalities. In case the points are not enough for the destruction of the province then the following cases are distinguished based on the comparison of the total points of the defending and the attacking units:

- If the total attack points are more than the defence points, then the defender loses all his defending personalities and the attacker loses followers or personalities whose power (attack) is in total equal to or greater than the difference of the attack and defence points.
- In case of a tie of the attack and defence points both players lose all their units.
- In case the defence points are more than the attack points, then the attacker loses all his units and the defender loses followers or personalities whose power (*attack*) is in total equal to or greater than the difference of the attack and defence points.

If the attacker survives the battle (and does not come out as the winner) then, after the followers or the personalities that are determined by the above rules die, the personalities that were used in the attack become tapped together with the followers that each personality has. The items (item) that each personality has lose one point of durability (durability).

If an item has zero durability (durability == 0) then it is destroyed and leaves the possession of the hero (detach). When a character dies then it disappears from the board of the game, while when a follower dies then it leaves the influence of the hero (detach). When a personality loses a battle and survives, it loses one honour point. When its honour points become zero then it is dishonoured and commits honourable suicide to save its family honour (*performSeppuku*).

### 2.4.4 Purchase phase

The player reveals his provinces and the relevant cards are printed (*economyPhase*). He can, by utilizing, as previously mentioned, his holdings, buy as many of the available cards as he wishes (i.e., either add a personality to his army or acquire a new holding). The player can buy only from the visible provinces. If he buys some card, he adds it to the corresponding area (the personality to the army and the holding to the holdings) and immediately replaces the empty spot with a card from the dynasty deck (but in a hidden state).

### 2.4.5 Final phase

Provided the player has more cards in his hand than the maximum limit, he must remove some of his cards until he reaches the allowed limit (*discardSurplusFateCards*). The cards that the player discards cannot be reused for the rest of the game. Finally, overall statistics for all the players are printed (*printGameStatistics, printHand, printProvinces, printArena, printHoldings*).

## 2.5 Winner of the game

As the winner of the game is defined the one who manages to destroy all the provinces of his opponents while he still has provinces. This condition is checked at the end of each round of a player (*checkWinningCondition*).

## 2.6 General comments

You are free to handle implementation or gameplay details as you wish e.g:
what will happen if the cards run out,
how cards “leave” the gameboard (simple change of isDead, transfer to a “graveyard” etc)
if you wish to add new cards or to skip some phase etc.

# *3. Files with Auxiliary Code*

For the determination of the statistics you have been given two reference files (Personalities_and_Holdings.txt, Followers_and_Weapons.txt) which contain statistics for each kind of card. Whatever statistic is not mentioned is left to your judgment to assign it a value. You are free to use whatever names or values you want.

Auxiliary code files are also provided that contain functions for the creation of the different decks (**DeckBuilder**) and for the conversion of object types (**TypeConverter**).

Provided you use them, you should be in a position to integrate them correctly into your modelling, but you can adapt them to your needs, adapt/add entities or methods and so on.

### 3.1 Deck Builder and general instructions.

The files **DeckBuilder.{cpp,hpp}** are a set of functions that allow you to construct and shuffle a deck automatically. It utilizes the constructors of the classes that you will construct and for this reason you must keep the names that have been given to you in the assignment.

Each constructor that you will implement must take two arguments in total. The one argument is the name of the card (**string**) and the other is its kind (**int**, via e.g. enumeration). The remaining statistics for each card should result according to its kind.

The statistics that have been given to you are purely indicative and you can replace them with your own, but in any case the restriction on the arguments of the constructor must be kept.

In order to be able to use the functions of the DeckBuilder you must, for compatibility reasons, define in your program the following enumerations in order to assign values to the types of each kind of card.

```
Personalities:
	enum{ATTACKER, DEFENDER, SHOGUN, CHANCELLOR, CHAMPION};
Holdings:
	enum{PLAIN, MINE, GOLD_MINE, CRYSTAL_MINE, FARMS, SOLO, STRONGHOLD};
Followers:
	enum{FOOTSOLDIER, ARCHER, SIEGER, CAVALRY, NAVAL, BUSHIDO};
Items:
	enum{KATANA, SPEAR, BOW, NINJATO, WAKIZASHI};
```

In the above enumerations SOLO corresponds to Gifts & Favors, FARMS corresponds to Farmland and NAVAL corresponds to Atakebune. These correspondences must be kept in order to be able to use the DeckBuilder.

### 3.2 TypeConverter and general instructions.

The files named **TypeConverter.{cpp,hpp}** are a set of functions that allow you to find the type of a card. It accepts three arguments: the first corresponds to the card that must be specialized and the other two are type pointers of each category. These pointers are initialized based on the result of the check.

For its operation, the implementation in the 4 classes of the function ***getType*** is required which returns an integer that corresponds to the category of the card and follows the enumerations below, storing the object in the appropriate pointer and setting the other one equal to NULL.

{PERSONALITY = 1, HOLDING, FOLLOWER, ITEM}

The above enumeration should preferably be kept throughout the program that you implement because in several cases you must have access to the category of each card.